# Probare: Solver-Consensus Verification and Predicated AVX-512 Lowering of Loops

**Srijan Mandal**

October 2026

## Abstract

Conditional loops over arrays are common in numerical and packet-processing code. On AVX-512 hardware a compiler that cannot prove the loads are safe may compute every lane and then blend, while the same loop can be written with opmask registers and no blend. This paper describes Probare, a verifier and code generator for such loops. Probare reads C or LLVM IR, turns each potentially undefined operation into an SMT query, and sends every query to seven SMT solvers; anything other than seven agreeing `unsat` answers stops the pipeline. For loops it can prove safe, it emits AVX-512 code that uses opmask registers, checks per lane that the vector code matches the scalar semantics, and signs the result in a 256-byte `.pch` header. I measured it on a Google Colab virtual machine with AVX-512, 100,000 runs per kernel. On an integer sparse reduction, the predicated code is no faster than scalar or clang's auto-vectorized code (704, 684 and 692 cycles at best). On a floating-point dot product it is 5.8× faster than the scalar loop (700 against 4,072 cycles at best), but its result is not bit-identical. The 99.99th-percentile latency is 33,000 to 41,000 cycles for every integer kernel, about fifty times the compute time. I also list what is not done: the seven solvers are not independent, no solver proofs are checked, contracts do not yet constrain anything, and the bare-metal runtime has no measurements.

# 1 Problem

## 1.1 Conditional loops and the blend

A common inner loop updates an accumulator only where a mask is set:

$$\mathbf{y}_i = \begin{cases} \mathbf{y}_i + (\mathbf{a}_i \times \mathbf{b}_i) & \text{if } \mathbf{m}_i \neq 0 \\ \mathbf{y}_i & \text{otherwise} \end{cases} \tag{1}$$

When a compiler cannot prove that the loads behind Eq. (1) are safe on masked-off lanes, it may fall back to speculative code: load and multiply every lane, build the predicate with a compare, then select between the old and new value with a blend (Figure 2). The speculative result stays live until the blend, the loads can fault on lanes the source never touched, and the dependency chain gets longer.

AVX-512 can do this directly. An opmask register selects the active lanes, masked-off lanes keep their old value, and faults on masked-off lanes are suppressed:

```
kmovw       %r8d, %k1                    # mask mi
vmovups     (%rdi), %zmm0{%k1}{z}       # ai (masked)
vmovups     (%rcx), %zmm1                # yi (
    accumulator)
vfmadd231ps (%rdx), %zmm0, %zmm1{%k1}  # yi += ai*bi
vmovups     %zmm1, (%rcx)                # store yi
```

Emitting this form is only legitimate if the loop's memory accesses are safe and the vector code means the same thing as the scalar loop. Probare's job is to establish both, and to leave a record that someone else can check.

## 1.2 Operating-system noise

Even a well-vectorized kernel shares its core with a kernel that takes interrupts. Scheduler ticks, inter-processor interrupts, page faults, TLB misses and last-level-cache contention all show up as latency spikes, and they dominate the tail whatever the compiler does. Section 6 measures how large that floor is on the test machine.

# 2 Probare

Probare is a Rust workspace of 12 crates. It takes C (compiled with `clang -O0 -Xclang -disable-O0-optnone -g` so that names survive) or textual LLVM IR for LLVM 18–22, and works on a bit-vector lowering of the IR rather than on the source.

## 2.1 What is supported

Supported: integer arithmetic with `nsw`/`nuw`, all `icmp` predicates, `phi`, `br`, `select`, `alloca`, `load`, `store`, `getelementptr` (including `inbounds` and multi-index struct and array offsets), vector operations, `llvm.fmuladd`, and 32- and 64-bit floating point. Non-recursive calls inside a module are inlined by substitution with fresh names. Recursion is rejected during CFG construction. Unsupported floating-point types are rejected with a diagnostic.

## 2.2 Memory

Memory is one array $\mathcal{M} : \text{Array}(\text{BV}_{64}, \text{BV}_8)$ with a parallel initialization bitmap $\text{Array}(\text{BV}_{64}, \text{BV}_1)$. A pointer is the concatenation of a 32-bit object id and a 32-bit offset, so pointers into different objects cannot alias by construction. Each `alloca` gets a fresh object id. A pointer parameter becomes an unconstrained symbolic object, with an assertion that distinct objects

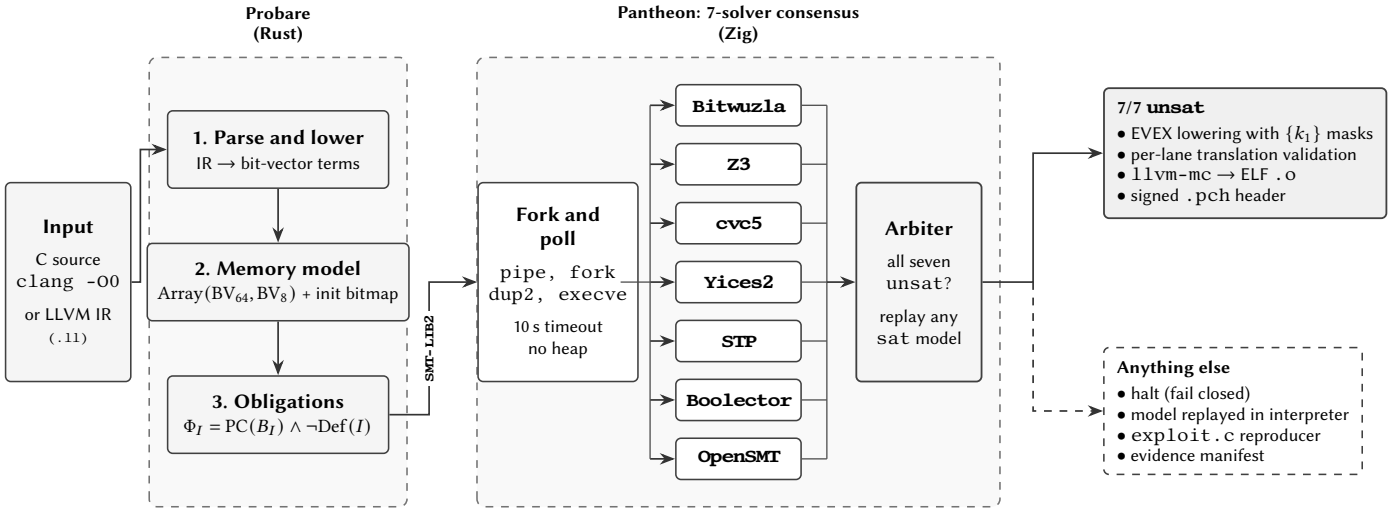



Figure 1: Probare pipeline. C or LLVM IR is lowered to bit-vector terms and a memory model, and each potentially undefined operation becomes one query $\Phi_I$. A Zig orchestrator runs all seven solvers on every query. Only seven agreeing `unsat` verdicts let the pipeline continue to code generation; any other outcome halts it.

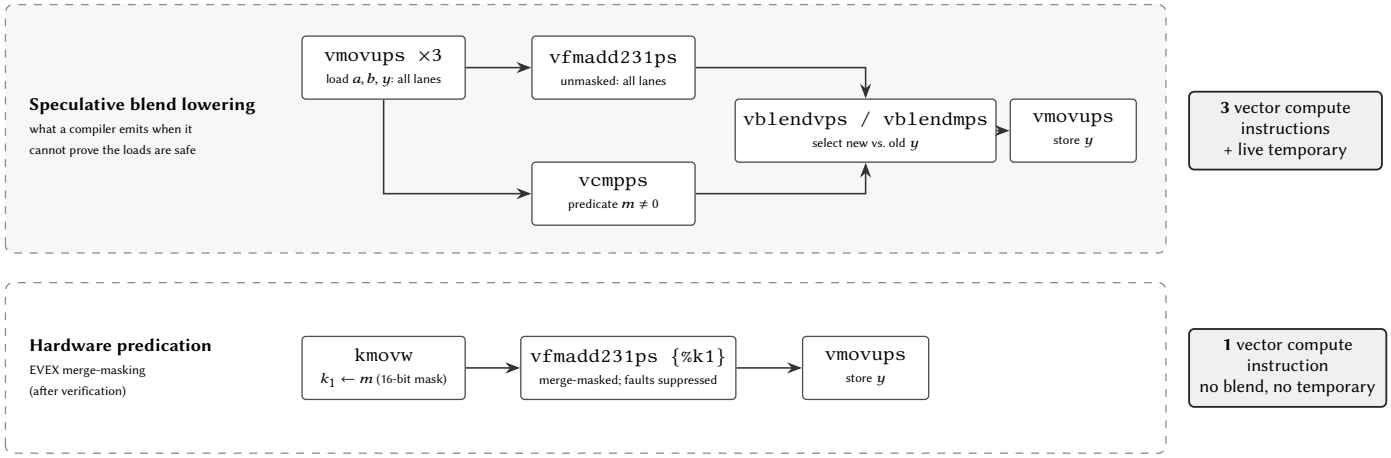



Figure 2: Static dataflow of Eq. (1). Top: compute everywhere, build the predicate, blend. Bottom: fold the predicate into the FMA through an opmask register. The counts are static instruction counts for the loop body, not measured $\mu$op counts.

are disjoint:

$$p_1 \neq p_2 \Rightarrow (p_1 + s \leq p_2) \vee (p_2 + s \leq p_1) \tag{2}$$

## 2.3 Obligations

Every instruction that can be undefined gets a definedness predicate $\mathrm{Def}(I)$ (Table 1). Probare emits one query per instruction:

$$\Phi_I = \mathrm{PC}(B_I) \wedge \neg\,\mathrm{Def}(I), \quad \mathrm{PC}(B) = \bigvee_{\pi \in \Pi(B)} \bigwedge_{c \in \pi} c \tag{3}$$

where $\Pi(B)$ is the set of paths from entry to $B$ and $c$ ranges over branch conditions on a path. $\Phi_I$ is `unsat` exactly when no feasible execution reaches $I$ with $\mathrm{Def}(I)$ violated. The path condition must be scoped to the instruction's own block: conjoining the conditions of several blocks is unsatisfiable for mutually exclusive branches and would certify everything. Before trusting an `unsat`, Probare also checks $\mathrm{PC}(B)$ itself; if it is unsatisfiable the block is reported as dead rather than certified.

| Instruction | $\mathrm{Def}(I)$ |
|---|---|
| `add`, `sub`, `mul` with `nsw`/`nuw` | no overflow, checked with sign or zero extension |
| `sdiv`, `udiv`, `srem`, `urem` | divisor $\neq 0$; for signed, not (INT_MIN, $-1$) |
| `shl`, `lshr`, `ashr` | shift amount < bit width |
| `load`, `store` | offset + size $\leq$ object size; for loads, bytes initialized |
| `unreachable` | $\mathrm{PC}(B) \Rightarrow$ *false* |

Table 1: Definedness conditions generated per instruction kind.

## 2.4 Loops

Probare has two ways to handle loops. The default is bounded model checking with counterexample-guided refinement [4, 5]: unroll the loop once, check whether any unwinding assertion can fail, and if so unroll further until the property is proved or a real refutation appears. For unbounded claims, loop transitions can be translated to constrained Horn clauses and solved by Z3's

Spacer engine [6] to find an inductive invariant.

Unrolling is not free, and I found that out the hard way (Section 3.4).

### 2.5 Floating point

Floating-point arithmetic has a QF_FPBV emitter (`fp.add`, `fp.fma`, round-to-nearest-even). Float literals are converted to their exact IEEE 754 bit patterns before entering the bit-vector terms. `llvm.fmuladd` is lowered to a multiply followed by an add; see Section 10 for why that matters when the target instruction is fused.

### 2.6 Contracts

A function can carry preconditions written as `__probare_requires(expr)` on the line of the definition or in the ten lines above it. The parser collects them into the function's `preconditions` list and the list is passed through unrolling and lowering. At the moment lowering adds a placeholder for each one rather than the constraint itself, so contracts are read but do not yet change any verdict.

### 2.7 Checking the encoding

Probare has a concrete interpreter for its IR. The encoding is tested by running the interpreter and the SMT lowering on the same randomized inputs and comparing results. This catches encoding mistakes but does not prove the encoding matches LLVM's reference semantics.

## 3 The solver consensus

A verifier that trusts one SMT solver inherits that solver's bugs; a wrong `unsat` on a satisfiable query certifies an unsafe program. The Pantheon is a Zig program that runs seven solvers on each query and accepts only if all seven agree.

### 3.1 Mechanics

For each SMT-LIB2 query the orchestrator creates a pipe per solver, forks, redirects with `dup2`, `execve`s the solver, and waits with `poll`. It uses fixed-size state and no heap allocator. Each child process gets a 2 GB address-space limit (`setrlimit`) before `execve`. At startup the command-line tool compares the SHA-256 of each solver binary with a `trusted_hashes.json` file and stops on a mismatch; if the file is absent, the check is skipped. Table 2 lists the solvers and versions.

### 3.2 Verdicts

Each solver ends in `unsat`, `sat`, `unknown`, a timeout (10 s per query) or a crash. Table 3 gives the arbiter's action. When a solver reports `sat`, Probare parses its model and replays it through the concrete interpreter. A model that does not reproduce is flagged as unconfirmed rather than trusted.

| Solver | Version | Notes |
|---|---|---|
| Bitwuzla [8] | 0.6.0 | `--lang=smt2` |
| Z3 [9] | 4.13.0 | `-in` |
| cvc5 [10] | 1.1.2 | CaDiCaL core |
| Yices 2 [11] | 2.6.4 | |
| STP [12] | 2.3.3 | Minisat core |
| Boolector [13] | 3.2.3 | Lingeling backend |
| OpenSMT [14] | 2.5.2 | QF_UF / QF_BV; arrays pre-expanded |

Table 2: The seven solvers. Solvers without native array support receive byte-offset formulas expanded by Probare.

| Outcome | Arbiter action |
|---|---|
| 7/7 `unsat` | Continue to code generation. |
| any `sat` | Report vulnerable (code `0x01`); replay the model; write a reproducer. Unconfirmed if replay fails. |
| timeout, crash, `unknown`, no consensus | Report divergence (`0x02`); reject. |

Table 3: Arbiter policy. Only unanimous `unsat` passes.

### 3.3 What the vote does and does not give

Let $W_i$ be the event that solver $i$ returns `unsat` on a satisfiable query. A false certificate needs all seven to be wrong at once, so

$$\Pr[\text{false cert.}] \le \min_i \Pr[W_i], \tag{4}$$

and the product $\prod_i \Pr[W_i]$ only if the failures are independent. They are not. Bitwuzla descends from Boolector, several solvers share SAT cores or preprocessing ideas, and OpenSMT reaches array queries through an expansion step that is itself code that can be wrong. The min-bound is the guarantee that holds unconditionally; the product is a hope. The vote is asymmetric in a useful way: a spurious `sat` can block a build but cannot let an unsafe one through.

The Pantheon produces votes and `sat` witnesses. There is an option to prepend `(set-option :produce-proofs true)` to a query and a small DRAT checking module, but proofs are off by default and none of the results in this paper use them.

### 3.4 Resource use

On 24 October 2026 the Linux OOM killer terminated a Probare run at 17.3 GB virtual and 13.6 GB resident memory. The run audited a sparse conditional-accumulation kernel with `--unwind-bound 16`. Each unrolled iteration produced dozens of obligations, and each obligation started seven solver processes. Concurrency was not bounded, so memory grew with unroll depth times obligations times seven.

The refutation was already found at $K = 1$, so unrolling to 16 added nothing. The audit script now runs at $K = 1$ and the run finishes in seconds. After this I added a 2 GB address-space limit per solver process, so one solver can no longer take the machine down. The number of solver processes running at once is still not capped, so a long unroll can still queue up a lot of work. A job queue with a global budget is the next change.

| Bytes | Field | Content |
|---|---|---|
| 0–3 | magic | 0x50434832, “PCH2” |
| 4–5 | version | 2 |
| 6–7 | isa_target | target ISA |
| 12–13 | solver_mask | advisory; 0x7F = all seven |
| 16–47 | ir_sha256 | digest of canonicalized IR |
| 48–79 | bin_sha256 | digest of emitted machine code |
| 192–255 | signature | Ed25519 over the header |

Table 4: Specified fields of the .pch v2 header. Remaining bytes are not described in this paper.

# 4 Predicated code generation

## 4.1 When a loop is lowered

A loop is lowered to predicated AVX-512 when (1) its condition can be turned into a vector mask, (2) its memory accesses are contiguous or provably disjoint (using the disjointness assertions above), and (3) its conditional effects fit opmask predication with $k_1$–$k_7$. The supported extensions are AVX-512F, VL and DQ. Probare does not emit a scalar tail loop: for $N$ not divisible by 16, the last iteration runs with

$$k = (1 \ll (N \bmod 16)) - 1 \tag{5}$$

so inactive lanes are masked off. Assembly is generated as text with {%k1} and {z} annotations and assembled by llvm-mc (x86-64, AVX-512 F/VL/DQ enabled) into an ELF object.

## 4.2 Translation validation

For each lowered loop, Probare builds an SMT query asserting that the vector code and the scalar IR disagree on some lane, and sends it to the same seven solvers. For every lane $i$ the claim is

$$\begin{aligned} k[i] = 1 &\Rightarrow v_{\text{evex}}[i] = v_{\text{scalar}}[i], \\ k[i] = 0 &\Rightarrow v_{\text{evex}}[i] = old[i] \end{aligned} \tag{6}$$

with $old[i]$ replaced by zero for {z} instructions. The query for the sparse-accumulate kernel is unsat. This covers the opcodes and the loop shape Probare emits today; it does not cover arbitrary AVX-512 code, and it trusts llvm-mc to assemble the text it is given.

# 5 The .pch header

Each emitted object comes with a 256-byte little-endian header (version 2). The fields that are fixed are in Table 4.

The IR digest is taken over a canonicalized, sorted form of Probare’s SSA IR so that whitespace and comments cannot change it. Signing uses Ed25519 [7] (ed25519-dalek); the private seed is meant to live in a hardware security module, and public keys are embedded in the verifier. A third party can check the header offline without running any solver, but then they are trusting the signer’s claim about the solvers. Re-running the evidence requires the evidence manifest that the audit writes next to the binary.

The solver_mask field is marked advisory: a loader that only checks the signature learns that the signer asserted seven-solver agreement, not that it happened. Figure 3 shows the checks a loader performs.

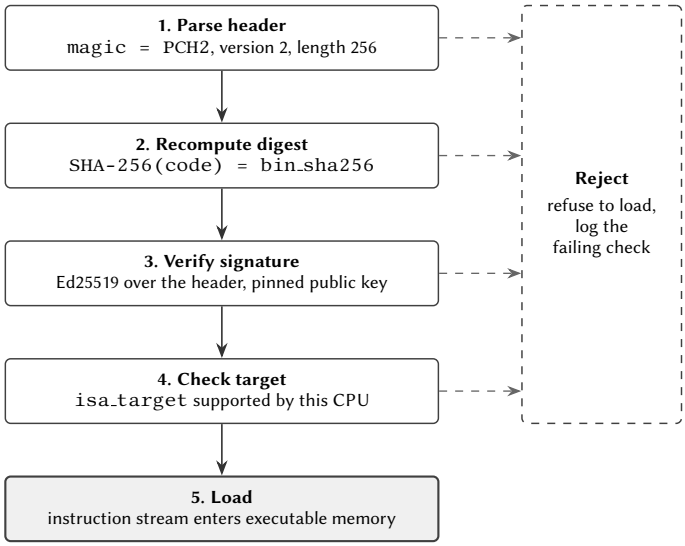



Figure 3: Loader-side checks for a .pch header. Any failure rejects the binary.

# 6 Evaluation

## 6.1 A kernel Probare refuses

The benchmark kernel sparse_accumulate adds src[i]*2.5f to dst[i] where mask[i] is set. Clang turns the update into llvm.fmuladd.f32. Running the audit at $K = 1$ ends in a refutation of obligation sparse_accumulate:for.body.0:4.0:Uninit, found by the seven-solver Pantheon, with a counterexample at $n$ = 2,147,483,647 and a generated reproducer (exploit.c).

That result is not a bug in a deployed system. dst, src and mask are raw pointer parameters, and Probare treats them as unconstrained, so it cannot show that reading mask[0] is initialized and in bounds. The kernel is only safe under a caller contract about buffer lengths and initialization, which the source does not state. What the tool did is report the missing assumption, with an input that violates it.

Translation validation is a separate check and passes on this kernel: the EVEX lowering is lane-wise equivalent to the scalar IR, and llvm-mc produces sparse_kernel_evex.o. These two results answer different questions. The first says the source needs a contract; the second says the vector code means what the scalar code means. A full pass needs both, and for this kernel only the second holds today.

## 6.2 Cycle measurements

The machine is a Google Colab virtual machine (a T4 GPU runtime; the GPU is not used). The CPU reports itself as an Intel Xeon at 2.00 GHz with the hypervisor flag set and AVX-512 F, DQ, CD, BW and VL. The compiler is clang 18.1.3 from Ubuntu 24.04. Each kernel ran 100,000 times, and times are TSC ticks read around each call. This is a shared virtual machine, so I could not control frequency, SMT or interrupts, and nothing here is a bare-metal result.

Table 5 has the results. The integer kernel is the sparse re-

| Kernel | MIN | MEAN | P99 | P99.99 | MAX |
|---|---|---|---|---|---|
| *Integer sparse reduction* | | | | | |
| Scalar baseline | 684 | 919 | 1,116 | 35,010 | 317,076 |
| Clang auto-vec | 692 | 973 | 1,234 | 38,946 | 231,154 |
| Probare predicated | 704 | 1,006 | 1,530 | 41,274 | 599,746 |
| *Floating-point dot product* | | | | | |
| Scalar baseline | 4,072 | 4,530 | 7,792 | 68,908 | 419,150 |
| Probare EVEX FMA | 700 | 779 | 1,090 | 33,120 | 48,580 |

Table 5: Cycles per call, 100,000 calls per kernel, Colab VM.

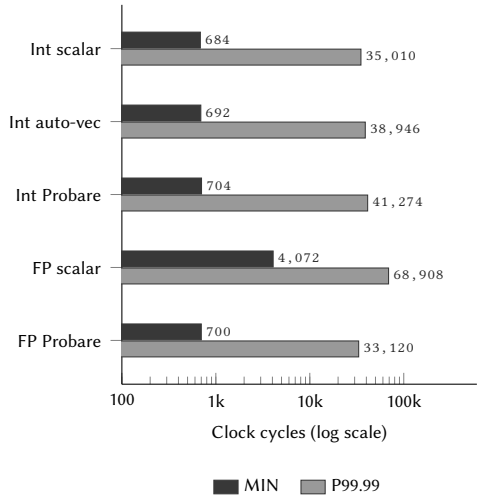



Figure 4: Best-case and 99.99th-percentile cycles from Table 5. The best case is about 700 cycles for four of the five kernels; the tail is about fifty times larger for all of them.

duction used throughout this paper; the floating-point kernel is a dot product on 32-bit floats. "Clang auto-vec" is the same C source compiled at `-O3` with AVX-512 and auto-vectorization enabled.

1. **Integer kernel: no gain.** The three versions are within 3% of each other at the minimum (684, 692, 704). Probare is slightly worse on the mean (1,006 against 919 and 973) and on P99 (1,530 against 1,116 and 1,234). All three return the same checksum, 52,445,175. On this machine and this kernel, predicated code does not beat what clang already produces.
2. **Floating-point kernel: 5.8×.** The predicated code needs 700 cycles at best against 4,072 for the scalar loop (5.8×), 779 against 4,530 on average, and its P99 and maximum are far lower. The two results are not identical: the checksums are 13,111,178 for scalar and 13,111,294 for Probare. The scalar loop adds in source order, which clang may not reorder without fast-math; the predicated version keeps 16 partial sums and fuses the multiply and add. So it is faster and also computes a different rounding of the sum. I have not separated how much of the 5.8× comes from predication and how much from reordering, and there is no auto-vectorized floating-point row to compare against.
3. **The tail is the same size for every kernel.** P99.99 is 33,000 to 41,000 cycles for the integer kernels, about fifty times the 700-cycle compute, and the maximum reaches 599,746. Bootstrap 95% intervals for P99.99 are [33,494, 44,110] for scalar and [34,070, 50,688] for auto-vectorized code. Probare's integer P99.99 of 41,274 lies inside both, so the differences between the integer kernels in the tail are not resolved by this run. Some of that noise comes from the hypervisor, and I have not tried to split it from guest-kernel noise.

# 7 Two systems-code audits

Probare is also used as a bug finder: when an obligation is `sat`, it writes a reproducer. I ran it on two ring-buffer designs.

## 7.1 `io_uring` ring masks

Completion-queue indexing is a power-of-two mask:

$$\text{index} = (\text{head} \,\&\, \text{ring_mask}) \ll \text{shift} \tag{7}$$

If `ring_mask` is immutable after setup, all seven solvers report `unsat` on the bounds obligation. If I instead model the mask as memory a concurrent user-space thread can write, all seven report `sat` in 0.109 s with

$$\text{head} = \text{ring_entries} = 1{,}048{,}576\ (\texttt{0x00100000}),$$
$$\text{corrupted_mask} = 4{,}294{,}967{,}295\ (\texttt{0xFFFFFFFF}),$$

giving an index of 1,048,576, past the end of the array. This is a statement about the modeled threat, in which the mask is read from writable shared memory. It does not by itself show a defect in a released kernel. Whether the real code reads the mask from such memory has to be established separately, and I do not claim a vulnerability here. The audited revision is [kernel version to be filled in].

## 7.2 DPDK `rte_ring`

The multi-producer enqueue computes

$$\Delta = (\text{prod_head} + n) - \text{cons_tail} \pmod{2^{32}} \tag{8}$$

For ring sizes $N \in \{64, 1024, 4096\}$, all seven solvers return `unsat` on the obligation that $\Delta \le$ capacity fails, including across the $2^{32}$ wrap. The result holds for those sizes and the sequential model of the update; it does not cover the full CAS retry protocol. The audited revision is [DPDK version to be filled in].

# 8 Two disclosures, and what they show

I filed two reports built from Probare obligations against real projects, through each vendor's own disclosure channel. The two outcomes are different in kind, and I report both as they actually closed rather than as I would like them to read.

## 8.1 Android Trusty TEE: confirmed, but not first

I reported an integer underflow in the VirtIO receive path of the Trusty TEE kernel (`lib/trusty/tipc_virtio_dev.c`,

handle_rx_msg): a 16-bit header length minus a larger computed shared-memory length underflows a 64-bit subtraction, and the subsequent bounds check is satisfied by the same modular wraparound that produced the bad value in the first place, so it does not catch it. Probare's model of this function refuted the safety obligation in 12 ms. Google's Android Security Team triaged the report, confirmed it against their own source, and closed it as a duplicate of an issue already known and fixed internally before my submission reached them — not eligible for a reward or CVE under their program rules, because duplication of a known issue is excluded regardless of how the finding was produced. Their own words: "this issue is a duplicate of a vulnerability previously reported internally… this issue was already known and fixed internally prior to your submission," and separately, "even though this specific issue was already known internally, your technical breakdown and analysis were very impressive."

I read this as a genuine positive result for the method, stated at the size it actually is: an independent, formal-verification-first path from LLVM IR to a real, previously-unpublished-to-me kernel bug, confirmed by the vendor's own engineers against their own source, that happened to have been found internally first. It is not evidence of a reward, a CVE, or priority, and I do not claim any of those.

### 8.2 Tesla TTPoE/modttpip: a real arithmetic flaw in code that is not in scope

I also modeled a length-validation gap in the modttpoe/modttpip kernel drivers (github.com/teslamotors/ttpoe) and reported it through Bugcrowd. Two things should be said about this one plainly. First, the write-up needed two rounds of self-correction: the initial arithmetic constants were wrong (corrected from 486 to 488 bytes, and from an 8-byte to a 36-byte available payload), and a claimed network-wide amplification via dev_queue_xmit was retracted once I checked skb_trim actually bounds the transmit path. A separate, narrower allocation/tail-pointer gap in ttpip_skb_aloc held up under the corrected model. Second, and decisively: Bugcrowd closed the report as Not Applicable because ttpoe/modttpip is not present on any Tesla asset in their program's scope — the code path the obligations were refuting is not reachable on a real target. Probare can be correct about the arithmetic of a function and that is a different claim from the function being attacker-reachable in a deployed system, and this report is the second case, not the first.

### 8.3 What this pair of outcomes argues

Together these are not a profitability or hit-rate claim; two disclosures is not a sample size for that, and I do not have a payout to report from either one. What they do show is a capability and a limit, side by side: Probare can find and precisely characterize a real, vendor-confirmed memory-safety bug directly from source with no fuzzing corpus and no execution on the target (Section 8.1), and a formal refutation of a model says nothing on its own about whether the modeled code is reachable, deployed, or in scope on a real system (Section 8.2). A verdict from Probare is a claim about the IR obligation as written, not a claim about a shipping attack surface, and confirming the latter is a separate step I did not automate and had to get wrong once before getting right.

| Tool | Emits EVEX masks | Formal proof | Solvers | Backend validated |
|---|---|---|---|---|
| LLVM / GCC | heuristic | no | — | no |
| CompCert [2] | no | Coq | — | yes (scalar) |
| Alive2 [1] | no | SMT | Z3 | IR only |
| Probare | yes | SMT votes | 7 | loop shapes it emits |

Table 6: Probare against related systems.

## 9 Related work

CompCert [2] proves semantic preservation in Coq for a scalar backend. It gives a proof, which Probare does not, and it covers a whole compiler, which Probare does not. Alive2 [1] validates LLVM optimizations at the IR level with Z3. It does not see the machine code. KLEE [3] explores paths dynamically and finds bugs but does not certify absence. Probare sits between them: it checks IR obligations and one backend step with several solvers, but without proof certificates and without covering a general compiler.

## 10 Limitations

1. **Baselines.** For the integer kernel, clang's auto-vectorized code is as fast as Probare's. For the floating-point kernel there is no auto-vectorized row, and clang cannot vectorize that loop without fast-math, so the 5.8× is against a loop it was not allowed to improve.
2. **One machine, two kernels.** All numbers come from one Colab virtual machine and two kernels. I do not know whether the results hold on bare metal or on other CPUs.
3. **Tail statistics.** With $10^5$ samples, P99.99 is about the tenth-largest sample and the bootstrap intervals are 10,000 to 16,000 cycles wide. Differences between kernels in the tail need $\geq 10^6$ runs.
4. **Noise floor.** The 33,000 to 41,000-cycle floor includes the hypervisor and is not decomposed. A tuning script (real-time scheduling, NMI watchdog off, hardware counters for context switches and cache misses) exists but has not been run, and a Colab VM would not allow most of it.
5. **Fused multiply-add.** llvm.fmuladd may be executed fused or unfused with different rounding. The translation validator still models the EVEX FMA as a multiply followed by an add; it should use fp.fma. Until it does, a rounding difference between the scalar and vector code would not be caught. The validator also checks the loop body lane by lane, not the order of the final reduction, which is where the floating-point checksums above differ.
6. **Solver independence and proofs.** The vote is not independent (Section 3) and no proofs are checked in the default path.

| Component | State | Missing |
|---|---|---|
| Rust frontend, obligations | works | contracts do not constrain yet |
| Seven-solver consensus | works | cap on concurrent solvers; proofs off by default |
| Solver hash check, 2 GB limit | works | hashes of the CLI itself |
| EVEX lowering, validation | works for sparse kernel | more kernels; `fp.fma` in validator |
| `.pch` v2, Ed25519 | works | enforcement of solver mask |
| Auto-vectorized baseline | timed, integer kernel | floating-point row |
| Tail statistics | run at $10^5$ samples | $10^6$ samples |
| Noise tuning | script written | a machine that allows it |
| Bare-metal runtime | small skeleton | measurements |

Table 7: What is built and what is missing.

7. **Contracts.** Raw pointer parameters are unconstrained. The contract syntax exists but does not yet constrain anything (Section 2).
8. **Resources.** Each solver is limited to 2 GB of address space, but the number of simultaneous solver processes is not capped.
9. **Trusted base.** The IR encoding (poison, `undef`, flags), the Zig orchestrator, the solver binaries, the OpenSMT array expansion, `llvm-mc`, the signing key and the CPU. The hash check covers the solver binaries only.
10. **Bare-metal runtime.** The runtime is a small freestanding x86-64 Zig program (`main.zig`, `x86.zig`) that reads the serialized TSC. It is a starting point, not a result; this paper has no measurements from it.

## 11 Status

The next steps, in order: find out why the predicated integer kernel is no faster than clang's, by reading the generated code of both; add an auto-vectorized floating-point row; repeat the runs on a machine where frequency, SMT and interrupts can be controlled, at $10^6$ samples; make the validator use `fp.fma`; turn contracts into real constraints; cap concurrent solver processes.

## 12 Conclusion

Probare checks a loop with conditional updates for undefined behavior with seven solvers, lowers it to masked AVX-512, validates the lowering lane by lane against the scalar form, and signs the result. The speed results so far are mixed. On a floating-point dot product the generated code is 5.8× faster than the scalar loop, partly because it reorders the sum. On an integer reduction it is no faster than clang's own output. On the Colab machine the tail latency is about fifty times the compute time for every kernel, which no code generator changes. The next work is to explain the integer result, to measure on a machine I control, and to turn the solver vote into something closer to a proof.

## References


[1] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr, "Alive2: Bounded Translation Validation for LLVM," in *Proc. PLDI*, 2021, pp. 65–79.

[2] X. Leroy, "Formal Verification of a Realistic Compiler," *Commun. ACM*, vol. 52, no. 7, pp. 107–115, 2009.

[3] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs," in *Proc. OSDI*, 2008, pp. 209–224.

[4] A. Biere, A. Cimatti, E. Clarke, and Y. Zhu, "Symbolic Model Checking without BDDs," in *Proc. TACAS*, 1999, pp. 193–207.

[5] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-Guided Abstraction Refinement," in *Proc. CAV*, 2000, pp. 154–169.

[6] A. Komuravelli, A. Gurfinkel, and S. Chaki, "SMT-Based Model Checking for Recursive Programs," in *Proc. CAV*, 2014, pp. 17–34.

[7] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *J. Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[8] A. Niemetz and M. Preiner, "Bitwuzla," in *Proc. CAV*, 2023, pp. 3–17.

[9] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Proc. TACAS*, 2008, pp. 337–340.

[10] H. Barbosa *et al.*, "cvc5: A Versatile and Industrial-Strength SMT Solver," in *Proc. TACAS*, 2022, pp. 415–442.

[11] B. Dutertre, "Yices 2.2," in *Proc. CAV*, 2014, pp. 737–744.

[12] V. Ganesh and D. L. Dill, "A Decision Procedure for Bit-Vectors and Arrays," in *Proc. CAV*, 2007, pp. 519–531.

[13] A. Niemetz, M. Preiner, and A. Biere, "Boolector 2.0," *J. Satisfiability, Boolean Modeling and Computation*, vol. 9, pp. 53–58, 2014.

[14] R. Bruttomesso, E. Pek, N. Sharygina, and A. Tsitovich, "The OpenSMT Solver," in *Proc. TACAS*, 2010, pp. 150–153.